Krishna Solanki

Event-Driven Microservices Review.pdf

Shri Vile Parle Kelavani Mandal

Document Details

Submission ID
trn:oid:::9832:154586668

Submission Date
Oct 14, 2026, 1:31 AM GMT+5:30

Download Date
Oct 14, 2026, 1:37 AM GMT+5:30

File Name
Event-Driven Microservices Review.pdf

File Size
185.5 KB

7 Pages

5,480 Words

32,795 Characters

63% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely revised by AI.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

24 AI-generated 63%
Likely generated or likely generated and revised by AI.

Disclaimer
Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (it may misidentify writing that is likely human generated as AI generated and likely AI generated as human generated) so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Safe Routing in Event-Driven Microservices: A Review of AIAER-Mask

Mr. Krishna Solanki
Dept. of Information Technology
NMIMS Mukesh Patel
School of Technology
Mumbai, India
krishna.solanki24@nmims.in

Mr. Chaitanya Tikku
Dept. of Information Technology
NMIMS Mukesh Patel
School of Technology
Mumbai, India
chaitanya.tikku42@nmims.in

Mr. Anuvrat Tripathi
Dept. of Information Technology
NMIMS Mukesh Patel
School of Technology
Mumbai, India
Anuvrat.tripathi13@nmims.in

Mr. Anay Modi
Dept. of Information Technology
NMIMS Mukesh Patel
School of Technology
Mumbai, India
Anay.modi60@nmims.in

Dr. Dharmesh Rathod
Dept. of Information Technology
NMIMS Mukesh Patel
School of Technology
Mumbai, India
Dharmesh.Rathod@nmims.edu

Abstract*—Event-driven microservices improve independent deployment and elastic scaling, but routing heterogeneous operations among replicas creates a conflict between low latency and semantic correctness. A fast endpoint may lack the authorization, data role, or operation capability required by an event, allowing performance-only selection to violate application intent even when requests complete successfully. This paper critically reviews AIAER-Mask, a neuro-symbolic router that separates deterministic admissibility checks from adaptive performance selection. We formalize its capability relation, hard action mask, fail-closed behavior, and constrained routing objective, and distinguish a genuine zero-probability safety guarantee from the merely small probability produced by a finite negative logit. We then examine the reported evaluation on the SocialNetwork and HotelReservation workloads in DeathStarBench, including workload-specific capability maps, baseline comparisons, failure injection, broker-delay emulation, and local region-like latency profiles. Hard-masked variants report complete intent correctness under validated active maps; unmasked baselines exhibit substantial invalid routing. The results do not establish universal latency superiority: masked heuristics are competitive, and the learned policy incurs additional inference and state-processing cost. The learned router shows its clearest benefit in bounded local region-latency emulation, not in a real multi-region deployment. A comparative case study maps the safety boundary to Saphire AOCS role-based access control, aircraft-to-gate wingspan checks, and database-backed session revocation. This mapping is architectural rather than an evaluated integration. Deterministic guards can make semantic constraints explicit and auditable, while offline reinforcement learning should optimize only within a maintained safe action set. Capability-map correctness, operational governance, tail latency, and production-scale validation remain essential open concerns.

***Index Terms*—Event-driven architecture, microservices, safe reinforcement learning, neuro-symbolic systems, intent correctness, action masking, airport operations.**

I. Introduction

Microservice architectures distribute application behavior across independently deployable services. Event-driven designs add asynchronous communication through brokers, decoupling producers from consumers and allowing services to scale or recover independently. A single user-visible operation may traverse several service boundaries, queues, and replicas. Consequently, routing influences not only response time and throughput, but also which component interprets and mutates application state. DeathStarBench made this systems problem experimentally accessible through end-to-end microservice workloads, including SocialNetwork and HotelReservation [2].

The resulting routing problem is not simply to select the least loaded endpoint. Replicas may differ in operation, authorization, data freshness, read/write role, or transactional capability. A replica with the best latency estimate is useful only when it is semantically admissible for the event at hand. For example, a router that forwards a state-changing operation to a read-only handler can violate the caller's intent even if the handler is healthy and fast. In a similarly incorrect route, sensitive work can cross a capability boundary. Such outcomes may present as application errors, silent omissions, stale state, or policy violations; performance metrics alone do not distinguish them.

This distinction is particularly important in event-driven systems. Broker delivery establishes that a message reached a consumer; it does not establish that the selected consumer is authorized or capable of performing the requested operation. Nor does adding replicas guarantee that those replicas are interchangeable. A routing policy that treats all endpoints as a single pool implicitly asserts such interchangeability, an assumption that fails for heterogeneous operation groups.

Hassan *et al.* introduce AIAER-Mask to separate the decisions "may this replica serve this operation?" and "which admissible replica is likely to perform best?" [1]. A deterministic capability map answers the first question; a learned offline discrete Soft Actor-Critic (SAC) policy ranks the remaining candidates using telemetry. The distinction is valuable because the safety condition is represented structurally, rather than entrusted to a reward weight. The paper reports experiments across two DeathStarBench workloads and provides careful qualifications on the limits of its evidence.

This review reconstructs the formal model, examines the architecture and experimental methodology, and evaluates what the results do and do not establish. It also relates the safety boundary to Saphire AOCS, a student-developed airport operations coordination platform described in the project context [14]. The comparison is an architectural synthesis, not evidence that AIAER-Mask has been integrated into or tested against Saphire. The central argument is that the symbolic guard is the primary correctness contribution; the adaptive policy is an optional performance mechanism whose value depends on workload dynamics, telemetry quality, and its operational cost.

II. Related Work and Routing Pitfalls

A. Heuristic Routing

Round-robin distributes requests according to a fixed sequence and is simple to audit, but it does not react to queue growth or heterogeneous service times. Least-connections and Power-of-Two-Choices (P2C) use load proxies to avoid some imbalance; P2C, for example, samples two choices and selects the less loaded one [3]. Exponentially weighted moving average (EWMA) routing adapts to observed latency, balancing responsiveness against the smoothing induced by its update factor. These heuristics are attractive where operational transparency and low computation are priorities.

However, none of these selection rules establishes that a candidate can correctly process a particular operation. Their safety depends on what enters the candidate set. If the router presents endpoints from multiple operation groups without a separate admissibility filter, round-robin can cycle into a wrong group, P2C can prefer a semantically invalid but lightly loaded replica, and EWMA can favor an invalid endpoint because its measured latency is low. The failure is not inherent to the heuristics: applying a capability filter before selection preserves their simple adaptive logic while removing invalid choices.

B. Reinforcement Learning and Offline Deployment

Reinforcement learning has been applied to scheduling, resource allocation, and service-instance selection, where policies can respond to interactions among load, latency, and resource contention [4], [5]. SAC optimizes expected return while incorporating policy entropy, and offline reinforcement learning (RL) learns from previously collected transitions rather than exploring directly in a live system [6]–[8]. Conservative Q-Learning, Implicit Q-Learning, and TD3+BC are examples of methods that address challenges in offline policy learning and distribution shift [9]–[11].

These techniques concern how a policy learns from data and how it performs with respect to a reward; they do not, by themselves, define the semantic validity of an action. A large penalty for an invalid choice may discourage it, but it remains a trade-off within the objective. Finite samples, imperfect reward weights, telemetry drift, or a learned policy's uncertainty may still make an invalid action attractive. Offline training removes live exploration during training, not errors in the training data, unsupported actions at inference time, or violations of a deployment capability rule.

C. Intent Correctness as a Distinct Property

Intent correctness should be reported separately from success rate and latency. A request may return successfully while reaching an endpoint that implements the wrong role or operation. Conversely, a fail-closed rejection for an unknown operation can preserve the routing boundary while reducing availability. Prior work on action masking distinguishes admissible from inadmissible choices during policy selection, while intent-based networking emphasizes translating goals into enforceable system behavior [12], [13]. AIAER-Mask applies these ideas to operation-to-replica routing: it treats capability as an explicit relation and masks the action space before a heuristic or learned selector makes its choice.

The practical implication is that a fair comparison must constrain every method to the same valid set when assessing safe performance. An unmasked policy may post lower latency simply because it is allowed to exploit replicas that should never serve the operation. Such a result is not an improvement over a safe method; it is an invalid comparison unless correctness is included as a first-class metric.

III. Mathematical Safety Formulation

Let $\mathcal{O}$ be the set of operation types and $\mathcal{A}$ the set of candidate service replicas or broker-subject endpoints. At decision time t, the router receives operation $o_t \in \mathcal{O}$, state s_t containing telemetry and deployment context, and selects action $a_t \in \mathcal{A}$. A deterministic capability function defines semantic admissibility:

$$C : \mathcal{A} \times \mathcal{O} \to \{0,1\}, \qquad C(a,o) = 1 \iff a \text{ may serve } o. \tag{1}$$

The valid-action set for an operation is

$$\mathcal{V}(o) = \{a \in \mathcal{A} : C(a,o) = 1\}. \tag{2}$$

Multiple replicas may be valid for one operation even if their queueing behavior, latency, success rate, or failure state differs. In the SocialNetwork configuration described by Hassan *et al.*, six operation groups map to 18 actions, with three valid replicas per operation [1].

For a routed operation with at least one valid replica, the per-decision safety requirement is $C(a_t, o_t) = 1$. Over an episode whose routed operations all have at least one valid replica, the corresponding probability-one property is

$$\Pr_{\pi}[\forall t,\ C(a_t, o_t) = 1] = 1. \tag{3}$$

If $\mathcal{V}(o_t) = \varnothing$, selecting a service endpoint cannot satisfy this requirement. The safe decision is therefore to reject or invoke an explicitly configured fallback, rather than infer a capability. Equivalently, one can extend the action space with a fail-closed action $\perp$ and define it as admissible only when no service action is valid. The paper's guarantee is conditional on a correct, active capability map and correct enforcement of that fallback behavior; it is not a proof that a real deployment's capability specification is itself correct.

The router seeks high cumulative reward while satisfying the hard safety condition. The constrained objective below applies to policy-selected service routes with nonempty valid sets; fail-closed cases are handled outside the learned selector:

$$\begin{aligned} \max_{\pi} \quad & \mathbb{E}_{\pi}\left[\sum_{t=0}^{\infty} \gamma^t r_t\right] \\ \text{subject to} \quad & \Pr_{\pi}[\forall t,\ C(a_t, o_t) = 1] = 1, \end{aligned} \tag{4}$$

where $0 \le \gamma < 1$ is the discount factor and r_t represents routing utility, including performance and reliability terms. Reward shaping can improve performance within the feasible region; it is not a substitute for the constraint in (4).

This distinction also sharpens the interpretation of a logit mask. Let $z_\theta(s, a)$ be the learned score for action a. The ideal hard mask is

$$M_a(o) = \begin{cases} 0, & C(a, o) = 1, \\ -\infty, & C(a, o) = 0. \end{cases} \tag{5}$$

The masked-logit computation can be expressed compactly as

$$\pi_\theta(\cdot \mid s, o) = \mathrm{softmax}_{a \in \mathcal{A}}(z_\theta(s, a) + M_a(o)). \tag{6}$$

Equivalently, its action probabilities are

$$\pi_\theta(a \mid s, o) = \begin{cases} \dfrac{\exp(z_\theta(s, a))}{\sum_{a' \in \mathcal{V}(o)} \exp(z_\theta(s, a'))}, & a \in \mathcal{V}(o), \\ 0, & a \notin \mathcal{V}(o). \end{cases} \tag{7}$$

This valid-set form, or an equivalent implementation that truly removes invalid choices, gives an exact zero probability to invalid actions. In finite-precision software, replacing $-\infty$ with a finite negative constant $-\kappa$ only makes that probability very small; it does not make it mathematically zero. A production implementation seeking the stated invariant should sample or maximize over $\mathcal{V}(o)$ directly, and should verify the empty-set branch independently.

IV. The AIAER-Mask Neuro-Symbolic Architecture

A. Safety and Adaptation Layers

AIAER-Mask places a deterministic symbolic intent guard in front of an offline discrete SAC router. The guard validates the operation-to-action capability map, constructs the admissible set, and handles unknown operations or empty sets through rejection or fallback. The neural component receives telemetry-derived state, including operation identity, latency estimates, queue-depth proxies, load, success rates, stress level, and replica metadata. It scores only the choices left available by the guard [1].

This division of responsibility is important. The capability map answers an application-level question and should be versioned as a deployment artifact. The learned policy answers a performance question and can be retrained or replaced without being allowed to redefine which operation a replica may perform. The mask is thus not a neural classifier or a reward feature; it is a control boundary.

B. Masked Policy and Fail-Closed Behavior

The policy's masked logits are often expressed as $z_\theta(s_t, a) + M_a(o_t)$ and normalized by a softmax. Equations (5) and (7) give the exact form of that operation. The capability check must run on every selection, not just during policy training. In addition, the capability map must correspond to the active deployment configuration. If an operation label is unknown, or all candidate replicas are unavailable, routing to a guessed endpoint would invalidate the guarantee; explicit rejection or a controlled fallback is required.

The article states a proposition that every selected non-fallback action is valid if the active map is correct, the mask is applied before selection, the selector operates only on masked actions, and unknown or empty action sets fail closed. The proof is direct: the chosen action is a member of $\mathcal{V}(o_t)$, which by definition implies $C(a_t, o_t) = 1$. This is a sound local argument about the selection rule, not machine-checked verification of the entire service deployment. Map authorship, runtime bypasses, stale policy state, and enforcement defects remain outside that proof.

C. Offline Learning and Stabilization

The adaptive component uses offline discrete SAC. Behavior-cloning regularization encourages agreement with logged behavior, but the reviewed design applies it only to logged actions valid under the active capability map. Regret-aware reward shaping estimates the latency gap between the selected action and the best valid replica when telemetry supports that comparison. These techniques target learning stability and ranking quality; the hard mask, rather than behavior cloning or shaping, supplies the semantic restriction.

The reported dataset pipeline is substantial but bounded. Baseline behavior logs cover seven policies under low, medium, and high stress with five seeds. The collection report records 210 CSV files and 27,227,991 loaded rows, from which a deterministic one-million-transition sample is used for training. The split is 70% training, 15% validation, and 15% test; final test measurements are not used for hyperparameter tuning. The article reports hashes for logs, configurations, datasets, and capability maps, as well as warm-up filtering and validation diagnostics. This is stronger experimental hygiene than reporting only a reward curve, although these records do not by themselves establish production reproducibility.

V. Empirical Evaluation on DeathStarBench

A. Workloads and Experimental Design

The evaluation uses Dockerized DeathStarBench SocialNetwork and HotelReservation applications with a NATS-based event fabric. Each workload has its own YAML capability map. For SocialNetwork, the validated map contains six operations, 18 actions, 18 valid operation-action pairs, and 90 invalid pairs. HotelReservation uses a separate map and operation structure, testing whether the same mask mechanism can be instantiated beyond one topology [1], [2].

The baseline set includes unmasked Static, EWMA, P2C, and SAC-style routing as well as masked variants such as Mask+Static, Mask+EWMA, Intent-safe P2C, and full AIAER-Mask. Additional comparisons include SAC without a hard mask and SAC with a soft intent penalty. The paper reports mean and tail latency, throughput, routing overhead, success rate, intent correctness, invalid-action rate, fallback counts, cumulative regret, optimal-valid-replica match, and adaptation or recovery time. This set correctly separates service-level success from whether a request was routed to an admissible replica.

The test scenarios include burst load, queue spikes, service-instance failures, degraded or invalid capabilities, stale maps, unknown operations, workload-mix shifts, and changing latency conditions. Offered-load tiers are configured at 5,000, 9,000, and 14,000 requests per second. Broker sensitivity uses added delays of 0, 2, 5, 10, and 20 ms. The study further emulates region-like latency profiles while services remain local. These are controlled experiments, not deployment tests across real brokers or geographic regions.

B. Intent Correctness and Baseline Comparison

Under validated active maps, hard-masked variants report 100% active capability validity and 0% invalid actions across the two workloads. Across comparable masked and unmasked pairs, masking improves mean intent correctness by 91.06 percentage points and reduces invalid-action rate by the same average amount. This is the strongest empirical contribution: it demonstrates the direct value of separating admissibility from optimization in the tested action spaces.

The unmasked results illustrate why raw speed must be interpreted cautiously. EWMA without a mask reaches only 17.39% intent correctness on SocialNetwork and 11.80% on HotelReservation, corresponding to invalid-action rates of 82.61% and 88.20%. Static without a mask is lower still on SocialNetwork, with 0.85% correctness and 99.15% invalid actions; on HotelReservation it reports 11.65% correctness and 88.35% invalid actions. A low-latency result from these configurations is not a safe-performance win because the methods can choose outside the operation's valid replica set.

The reported mean latency for full AIAER-Mask is 40.78 ms on SocialNetwork and 40.68 ms on HotelReservation, while preserving 100% correctness. Intent-safe P2C is the fastest listed masked method on average across both workloads; masked EWMA and other safe heuristics remain competitive. Thus, the evidence supports the mask as a reusable safety boundary more strongly than it supports the learned router as a generally faster selector.

C. Dynamic Scenarios, Failures, and Delay Sensitivity

In the controlled dynamic scenarios, the best valid replica changes over time, and the reported average latency gap between the best and worst valid replicas ranges from approximately 31 to 231 ms by scenario. Nevertheless, Mask+EWMA is the strongest intent-safe dynamic baseline by mean latency, cumulative regret, and optimal-replica match in the current scenarios. This is a useful critical result: non-stationarity alone does not establish that an RL policy will outperform a well-tuned telemetry heuristic.

The clearest adaptive benefit appears in Local Region-Latency Emulation. Full AIAER-Mask reports the lowest cumulative regret among intent-safe variants at 63.95 ms and a 98.29% optimal-valid-replica match. Relative to Mask+EWMA, the article reports approximately 93.1% lower regret and a 4.07 percentage-point improvement in optimal match. These results indicate value under the paper's bounded timing emulation; they do not demonstrate readiness for real multi-region traffic. In the same setting, unmasked EWMA has lower raw latency but only 16.12% intent correctness and an 83.88% invalid-action rate.

Failure injection preserves 100% correctness and zero invalid actions for known operations under the active map across burst load, queue-depth spikes, service failures, and network-latency spikes. Unknown operations take a separate rejection/fallback path. The paper reports 100% fallback success for fallback requests but approximately 84.95% overall success/intent-correctness in that unknown-operation scenario. The latter must not be paraphrased as a failure of the mask: unknown labels are deliberately not considered ordinary valid operations, and safe rejection trades request availability for prevention of an unsupported route. It does, however, show why fallback behavior and service success need independent reporting.

Broker-delay emulation leaves intent correctness unchanged, as expected when only timing changes and the active map remains intact. Delays up to 10 ms are partly absorbed by queueing and measurement variation; 20 ms produces a clearer latency increase and throughput degradation. Since the broker is not swapped and no geographically distributed deployment is used, these results establish sensitivity to injected delay rather than comparative broker or multi-region behavior.

D. Overhead and Evidence Quality

The article distinguishes the cheap symbolic mask from the added cost of the learned policy. It reports an average overhead change of 1.548 μs per request for simple masked heuristic comparisons and 335.190 μs per request for AIAER/SAC hard-mask comparisons. For full AIAER-Mask, the operational report gives mean inference overhead of 367.83 μs per request after warm-up, with p50, p95, and p99 values of 21.50, 1034.50, and 1287.60 μs, respectively. The wide spread underscores that average overhead alone is insufficient

TABLE I
SELECTED CROSS-WORKLOAD RESULTS REPORTED BY HASSAN ET AL. VALUES ARE CONDITIONAL ON THE PAPER'S VALIDATED, WORKLOAD-SPECIFIC CAPABILITY MAPS.

Router / Workload	Mean Latency	Intent Correctness	Invalid Actions	Interpretation
Full AIAER-Mask, SocialNetwork	40.78 ms	100%	0%	Safe learned router
Full AIAER-Mask, HotelReservation	40.68 ms	100%	0%	Safe learned router
EWMA, no mask, SocialNetwork	Not reported here	17.39%	82.61%	Performance-only selection
EWMA, no mask, HotelReservation	Not reported here	11.80%	88.20%	Performance-only selection
Static, no mask, SocialNetwork	Not reported here	0.85%	99.15%	Highly unsafe in this setup
Static, no mask, HotelReservation	Not reported here	11.65%	88.35%	Highly unsafe in this setup

when routing participates in a latency-sensitive request path. The mask-rebuild microbenchmark itself averages 0.0026 ms; the reported model checkpoint is approximately 914 KB and selected-run training time is 66.98 s [1].

The experiments are strengthened by seeded runs, warm-up filtering, capability-map validation, held-out diagnostics, and explicit labels on emulated conditions. Some reported tables in the source present different aggregation scopes, so values should be interpreted with their stated workload and regime rather than combined as if they came from one homogeneous trial. A remaining limitation is that benchmark correctness under a supplied map cannot establish that a production map captures every business, security, or consistency invariant.

VI. COMPARATIVE CASE STUDY: SAPHIRE AOCS

Saphire AOCS is an airport operations coordination project with airside conflict and telemetry components, turnaround task orchestration, baggage-event reconciliation, role-based operational dashboards, and a state-backed authentication session table. The supplied project context describes a Spring Boot and Spring Security backend, PostgreSQL persistence, and role-gated operational endpoints [14]. The following comparison maps concepts, not implementations: the project documentation does not state that an RL router or AIAER-Mask component is deployed in Saphire.

The closest correspondence is between the capability predicate and role-based access control (RBAC). A Saphire endpoint guarded by `@PreAuthorize` admits a role-specific operation; the analog of $C(a, o)$ is an auditable policy that relates an actor and requested operation to the endpoint or action. In a routing design, this check should be enforced server-side on every state-changing path rather than trusted to a front-end dashboard.

The analogy to a hard action mask is broader than RBAC. A gate-assignment rule that rejects an aircraft whose wingspan exceeds a stand's supported envelope is a domain-integrity predicate: the proposed action is ineligible regardless of its scheduling score. Likewise, checking `auth_sessions` supports immediate revocation or invalidation of a formerly valid token. The session check is not itself a replica capability map; it supplies current authorization state that must be consulted before a protected action proceeds. Keeping these distinctions explicit prevents the analogy from conflating authentication, authorization, physical constraints, and routing.

For a protected dispatch, these safeguards can be composed as a conceptual admissibility predicate:

$$M(u, o, f, g, a) = S(u) \wedge R(u, o) \wedge W(f, g) \wedge C(a, o), \quad (8)$$

where $S(u)$ denotes a currently valid session for user u as established through `auth_sessions`, $R(u, o)$ denotes the role authorization enforced by `@PreAuthorize` for operation o, $W(f, g)$ denotes the physical compatibility of flight f and gate g, and $C(a, o)$ denotes the capability of endpoint a to serve the operation. The first three predicates represent Saphire-side identity, authorization, and operational-integrity checks; the last is the routing admissibility relation formalized earlier. A scheduler may rank alternatives only after these conditions hold. This composition is a design synthesis, not an implementation or tested integration in Saphire AOCS.

Saphire telemetry suggests a plausible optimization layer: runway queue length, apron occupancy, flight delay, and turnaround milestones could help rank already permissible staff or equipment assignments. That is conceptually similar to AIAER-Mask's telemetry-conditioned policy, but the project context describes dynamic dispatch as a system requirement, not a tested SAC scheduler. The engineering lesson is therefore architectural: retain deterministic, testable authorization and domain guards as the outer boundary, and allow adaptive scheduling to optimize only among assignments that pass those guards. For an airport operations system, fail-closed behavior, human escalation, audit logs, and rollback deserve explicit treatment because a software rejection may itself affect operations.

TABLE II
ARCHITECTURAL CORRESPONDENCE BETWEEN AIAER-MASK AND SAPHIRE AOCS. THE MAPPING IS A DESIGN ANALOGY, NOT A CLAIM OF INTEGRATED OR CERTIFIED DEPLOYMENT.

Concern	AIAER-Mask	Saphire AOCS Analogue
Capability admissibility	Versioned relation between operation and eligible replica	Spring Security role checks such as `@PreAuthorize` on operational endpoints
Hard domain constraints	Mask excludes replicas that cannot serve the requested operation	Aircraft wingspan-to-gate compatibility checks reject physically unsuitable assignments
Authentication state	Active capability map participates in routing authorization	Database-backed `auth_sessions` permits session invalidation in addition to token validation
Telemetry and adaptation	Latency, queue, load, success, stress, and replica features guide ranking	Runway queues, apron occupancy, turnaround milestones, and flight-status telemetry inform dispatch decisions
Separation of concerns	Symbolic layer enforces admissibility; learned layer optimizes inside it	Authorization and transactional/domain integrity remain independent of scheduling and dashboard adaptation

VII. CRITICAL DISCUSSION, TRADE-OFFS, AND LIMITATIONS

A. What the Guarantee Does and Does Not Cover

Hard masking changes intent correctness from a preference into an invariant of the action-selection path, but only under stated assumptions. The active capability map must be complete and current; the operation identifier must be trustworthy; the mask must be applied on every path; and failures must not bypass enforcement. A stale map can be internally valid yet wrong for the live service topology. The source paper tests stale-map scenarios relative to an active map and emphasizes that active-map correctness is an assumption, not something the neural policy can supply. Capability-map ownership, review, schema validation, CI checks, canary rollout, and rollback are therefore safety-critical operations.

The guarantee is also about the selected endpoint, not the correctness of that endpoint's code, its downstream calls, or the business transaction it performs. A valid replica may be unhealthy, stale, compromised, or inconsistent. Additional mechanisms such as authorization at the service boundary, idempotency controls, transactional guarantees, and end-to-end audit remain necessary. In a distributed event system, consumer retries and duplicate delivery are separate correctness concerns that an action mask does not solve.

B. Offline RL Complexity and Deployment Cost

Offline training avoids live exploratory actions but introduces dataset and lifecycle requirements. The policy can only learn ranking behavior supported by its logs; changes in topology, operation mix, queueing regime, or telemetry semantics can create distribution shift. Behavior cloning and regret shaping may improve learning behavior, but neither eliminates this risk. Safe rollout consequently requires model and data versioning, held-out evaluation, shadow or canary deployment, runtime monitoring, retraining triggers, and a fallback heuristic. These are established machine-learning operations concerns rather than incidental implementation details [15], [16].

Reported inference cost also complicates adoption. The symbolic mask is extremely inexpensive, but full AIAER-Mask has a long-tailed per-request overhead distribution in the reported environment. A serving path with a strict tail-latency budget should compare end-to-end latency at matched correctness levels, include feature-construction and telemetry costs, and consider batching, caching, or optimized inference. If masked EWMA or P2C already meets objectives, the learned router's additional complexity may not be justified.

C. Generalization Boundaries and Operational Abstention

The evaluation covers two benchmark applications, controlled failure scenarios, local broker-delay emulation, and region-like timing profiles injected while services remain local. It does not include production-derived traces, real multi-broker experiments, deployment across regions, or a broad evaluation of service-specific map evolution. Therefore, the authors' cautious framing is appropriate: current evidence supports cross-workload instantiation and bounded timing sensitivity, not a universal claim of superior latency or production-grade multi-region generalization.

The fail-closed branch creates an availability trade-off. Rejecting an unknown operation prevents an unsafe guess but can interrupt workflow. A practical design needs a specified fallback with explicit semantics, bounded retries, observability, and a human or service-owner path to resolve new capabilities. Automatically promoting an unknown action into the active map would convert an auditable configuration decision into an unreviewed learned decision and weaken the guarantee.

D. Future Work: In-Kernel and Hardware-Assisted Guards

One future direction is to evaluate whether a compact capability check can be enforced closer to the network or service ingress, for example with an eBPF program using a userspace-maintained map. Linux BPF maps are mechanisms for sharing state between kernel and userspace, but that implementation choice does not itself establish semantic correctness or guarantee hardware offload [17]. A rigorous prototype would need to measure verifier constraints, atomic map updates, stale-map

behavior, enforcement coverage, and tail latency. Hardware offload should be treated as a separate platform capability with its own validation, rather than assumed from the use of eBPF. The neural ranking layer could remain in userspace while the deterministic guard provides a narrow, independently testable enforcement point. This is a research proposal; the reviewed AIAER-Mask experiments do not evaluate eBPF or hardware acceleration.

VIII. Conclusion

This review examined safe intent routing as a constrained decision problem in which endpoint eligibility precedes performance ranking. AIAER-Mask makes this separation explicit through a deterministic capability map, hard action masking, fail-closed handling, and an offline discrete SAC router. Under the active validated maps in the reported SocialNetwork and HotelReservation tests, masked variants achieve 100% intent correctness and zero invalid actions, while unmasked baselines often route incorrectly at high rates. These results support hard masking as a reusable software-engineering pattern.

The evaluation also counsels against equating the learned policy with the contribution. Masked heuristics remain competitive and sometimes outperform AIAER-Mask in stable or controlled dynamic regimes. The learned policy's clearest reported gain occurs under local region-latency emulation, and its inference overhead and operational lifecycle impose costs. Neither the safety claim nor the performance evidence extends beyond the validated active maps and tested scenarios.

The Saphire AOCS comparison reinforces a practical lesson: RBAC, session revocation, physical compatibility rules, and transactional invariants should be deterministic and auditable, while adaptation should select only among valid actions. More broadly, systems teams should report correctness, availability, latency, and fallback behavior separately; version and govern capability maps as executable policy; and validate the full production path before describing a safety property as guaranteed. Neuro-symbolic routing is valuable not because learning replaces rules, but because learning is confined by rules whose assumptions remain visible and testable.

References

[1] H. Hassan, S. Sarhan, and A. Shalaby, "AIAER-Mask: Safe neuro-symbolic intent-correct routing for event-driven microservices," *IEEE Trans. Services Comput.*, vol. 19, no. 5, pp. 3336–3346, Sep./Oct. 2026, doi: 10.1109/TSC.2026.3708565.

[2] Y. Gan *et al.*, "An open-source benchmark suite for microservices and their hardware-software implications for cloud and edge systems," in *Proc. 24th Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2019, pp. 3–18, doi: 10.1145/3297858.3304013.

[3] M. Mitzenmacher, "The power of two choices in randomized load balancing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 12, no. 10, pp. 1094–1104, Oct. 2001.

[4] W. Boudieb, A. Malki, M. Malki, A. Badawy, and M. Barhamgi, "Microservice instances selection and load balancing in fog computing using deep reinforcement learning approach," *Future Gener. Comput. Syst.*, vol. 156, pp. 77–94, 2024, doi: 10.1016/j.future.2024.03.010.

[5] D. Wu *et al.*, "Reinforcement learning for communication load balancing: Approaches and challenges," *Front. Comput. Sci.*, vol. 5, Art. no. 1156064, 2023.

[6] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proc. 35th Int. Conf. Machine Learning*, vol. 80, 2018, pp. 1861–1870.

[7] S. Levine, A. Kumar, G. Tucker, and J. Fu, "Offline reinforcement learning: Tutorial, review, and perspectives on open problems," arXiv:2005.01643, 2020.

[8] R. F. Prudêncio, M. R. O. A. Máximo, and E. L. Colombini, "A survey on offline reinforcement learning: Taxonomy, review, and open problems," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, pp. 10237–10257, Aug. 2024, doi: 10.1109/TNNLS.2023.3250269.

[9] A. Kumar, A. Zhou, G. Tucker, and S. Levine, "Conservative Q-learning for offline reinforcement learning," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 1179–1191.

[10] I. Kostrikov, A. Nair, and S. Levine, "Offline reinforcement learning with implicit Q-learning," in *Proc. Int. Conf. Learning Representations*, 2022.

[11] S. Fujimoto and S. Gu, "A minimalist approach to offline reinforcement learning," in *Advances in Neural Information Processing Systems*, vol. 34, 2021, pp. 20132–20145.

[12] S. Dey, S. Bhat, P. Dasgupta, and S. Dey, "Imperative action masking for safe exploration in reinforcement learning," in *Proc. Int. Workshop Explainable, Transparent Autonomous Agents and Multi-Agent Systems*, 2023, pp. 130–142, doi: 10.1007/978-3-031-40878-6_8.

[13] A. Leivadeas and M. Falkner, "A survey on intent-based networking," *IEEE Commun. Surveys Tuts.*, vol. 25, no. 1, pp. 625–655, 1st Quart. 2023, doi: 10.1109/COMST.2022.3215919.

[14] K. Solanki, C. Tikku, A. Tripathi, and A. Modi, "Saphire AOCS: Practical system context and comparative case study," unpublished course-project documentation, Software Engineering, SVKM's NMIMS MPSTME, Mumbai, India, 2026.

[15] D. Kreuzberger, N. Kühl, and S. Hirschl, "Machine learning operations (MLOps): Overview, definition, and architecture," *IEEE Access*, vol. 11, pp. 31866–31879, 2023, doi: 10.1109/ACCESS.2023.3262138.

[16] D. Sculley *et al.*, "Hidden technical debt in machine learning systems," in *Advances in Neural Information Processing Systems*, vol. 28, 2015, pp. 2503–2511.

[17] Linux Kernel Documentation, "BPF maps," accessed Oct. 13, 2026. [Online]. Available: https://docs.kernel.org/bpf/maps.html.